



Adversarial Attacks against Machine Learning

Gilad Gressel
Amrita School of Engineering

Introduction

- Definition
- Demo Evasion Attacks
- Who is Affected



Place your
Webcam Video here
Size 100%



Adversaries

- What is an adversary?
- An enemy
- Anybody who is trying to obstruct you



Is Machine Learning Secure?



- We can use machine learning to automate so many difficult tasks, including many cybersecurity problems.
- But when machine learning *becomes* the security we have to ask – is machine learning secure?
- The answer is “no”



Adversarial Attacks on ML

- Turns out that ML itself is very easy to break



x

“panda”

57.7% confidence

$+ .007 \times$



$\text{sign}(\nabla_x J(\theta, x, y))$

“nematode”

8.2% confidence

$=$



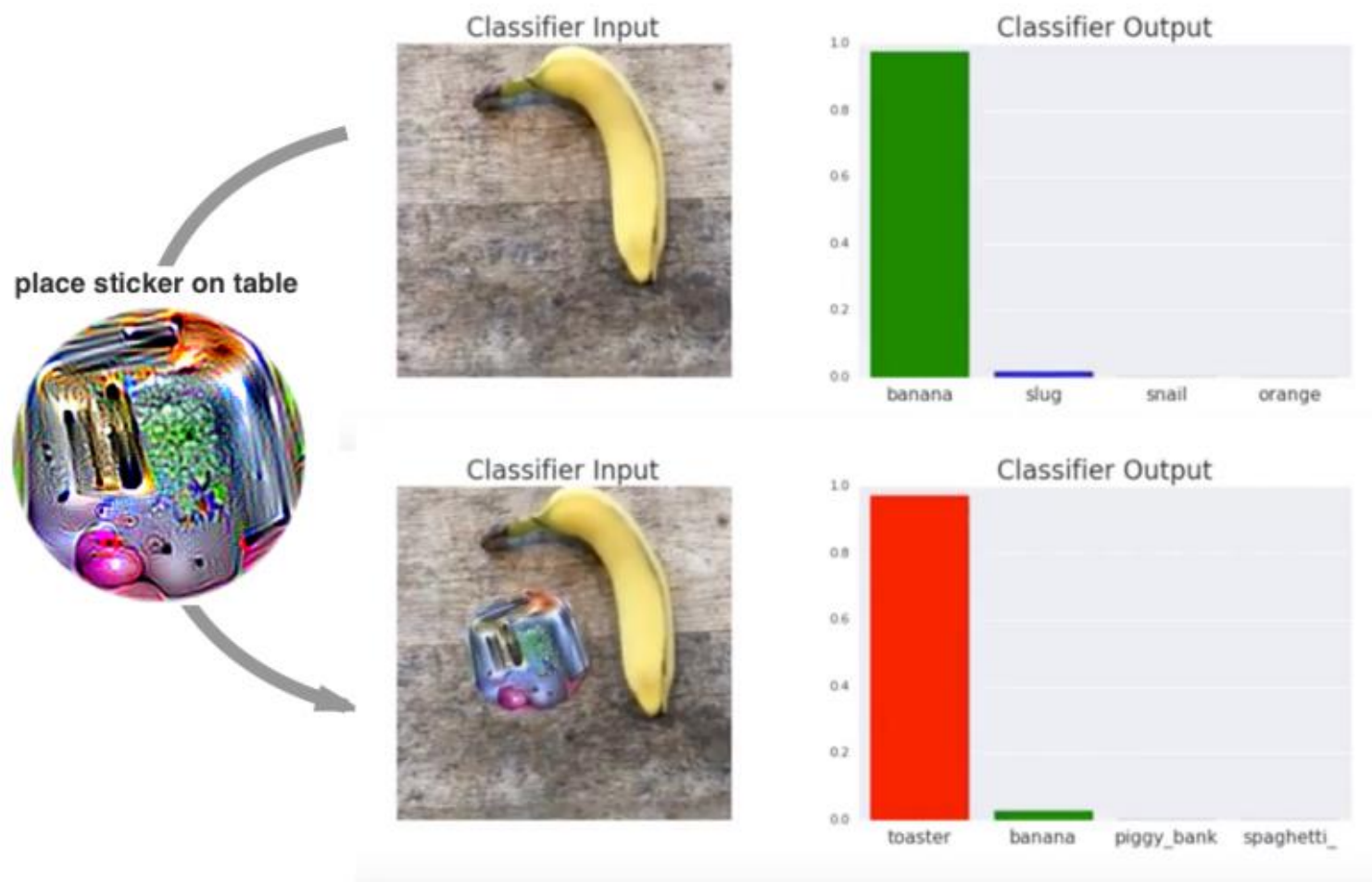
$x + \epsilon \text{sign}(\nabla_x J(\theta, x, y))$

“gibbon”

99.3 % confidence

Goodfellow, Ian J., Jonathon Shlens, and Christian Szegedy. "Explaining and harnessing adversarial examples." *arXiv preprint arXiv:1412.6572* (2014). – **7642 citations!!**

Adversarial Patch - Printable



- Brown, Tom B., et al. "Adversarial patch." *arXiv preprint arXiv:1712.09665* (2017).



Adversarial Patch - Video



Adversarial Objects



<https://arxiv.org/pdf/1707.07397.pdf>

Adversarial Sunglasses

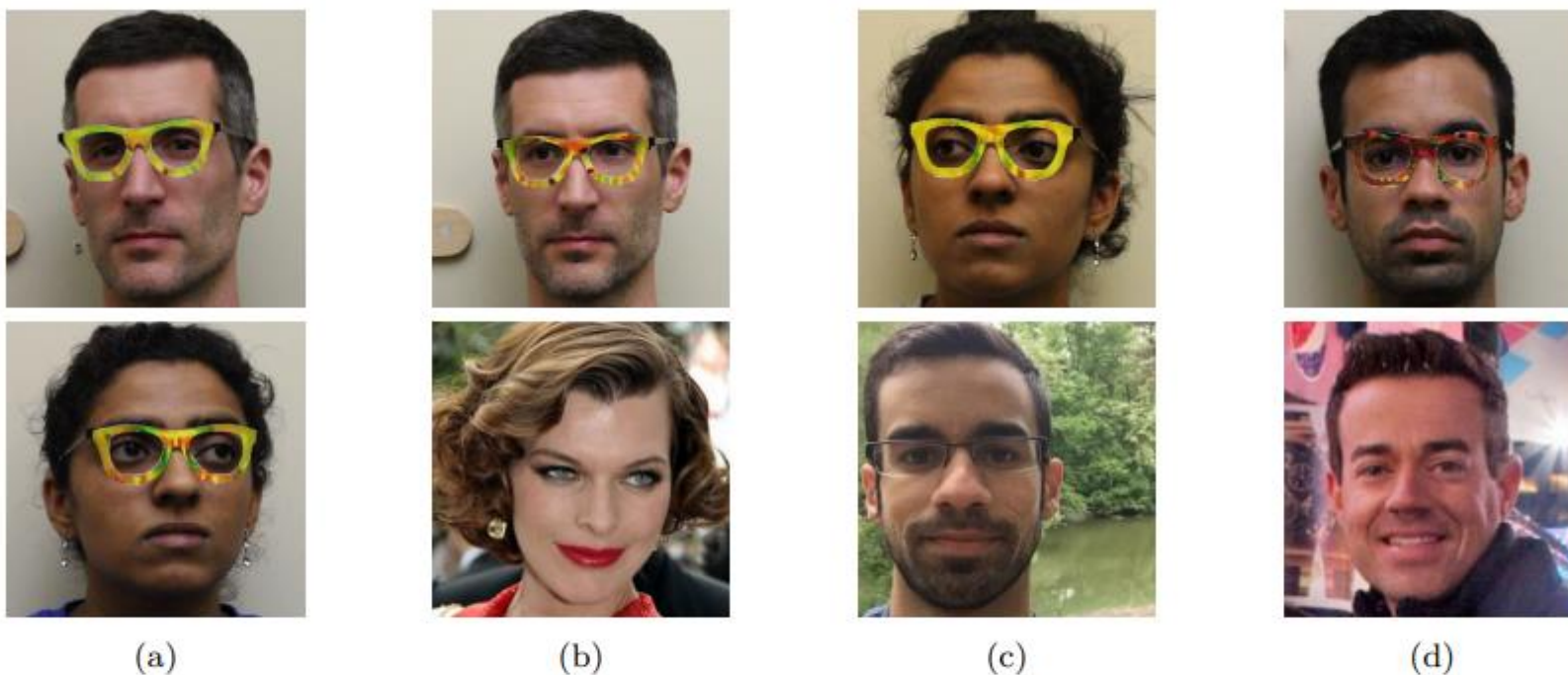


Figure 4: Examples of successful impersonation and dodging attacks. Fig. (a) shows S_A (top) and S_B (bottom) dodging against DNN_B . Fig. (b)–(d) show impersonations. Impersonators carrying out the attack are shown in the top row and corresponding impersonation targets in the bottom row. Fig. (b) shows S_A impersonating Milla Jovovich (by Georges Biard / CC BY-SA / cropped from <https://goo.gl/GlsWIC>); (c) S_B impersonating S_C ; and (d) S_C impersonating Carson Daly (by Anthony Quintano / CC BY / cropped from <https://goo.gl/VfnDct>).

<https://users.ece.cmu.edu/~lbauer/papers/2016/ccs2016-face-recognition.pdf>

What Domains are Affected?

- Most Popular: Computer Vision
- Malware
- Spam
- Phishing
- Voice
- Text
- And more



Conclusion

- Adversarial attacks attempt to thwart machine learning models
- They exist in pretty much all domains of ML

Place your
Webcam Video here
Size 100%



How Adversarial Evasion Attacks are Performed

Gilad Gressel
Amrita School of Engineering

Introduction

- Fast Gradient Sign Method
- Mimicry Attacks



Place your
Webcam Video here
Size 100%



FGSM – Fast Gradient Sign Method

EXPLAINING AND HARNESSING ADVERSARIAL EXAMPLES

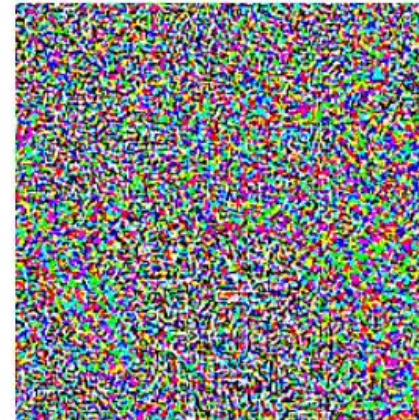
Ian J. Goodfellow, Jonathon Shlens & Christian Szegedy
Google Inc., Mountain View, CA
{goodfellow, shlens, szegedy}@google.com

 x

“panda”

57.7% confidence

+ .007 ×

 $\text{sign}(\nabla_x J(\theta, x, y))$

“nematode”

8.2% confidence

=

 $x +$ $\epsilon \text{sign}(\nabla_x J(\theta, x, y))$

“gibbon”

99.3 % confidence

- How do we create the perturbation?

Adversarial Perturbations



- Recall $y=mx+c$, and gradient descent
- We optimize m and c , in order to minimize a loss function (we used MSE)
- This results in the best m and c to *fit* the data
- Adversarial evasions generally do the same thing, except they optimize the *input* (image) to *maximize* the loss function
- This has the effect of making the input more *unlike* the target value
- You are basically trying to turn your input into something else
 - but crazily enough, it *looks* the same to the human eye!

Basic Optimization Idea

EXPLAINING AND HARNESSING ADVERSARIAL EXAMPLES

Ian J. Goodfellow, Jonathon Shlens & Christian Szegedy
Google Inc., Mountain View, CA
{goodfellow, shlens, szegedy}@google.com

Let θ be the parameters of a model, x the input to the model, y the targets associated with x (for machine learning tasks that have targets) and $J(\theta, x, y)$ be the cost used to train the neural network. We can linearize the cost function around the current value of θ , obtaining an optimal max-norm constrained perturbation of

$$\eta = \epsilon \text{sign}(\nabla_x J(\theta, x, y)).$$

We refer to this as the “fast gradient sign method” of generating adversarial examples. Note that the required gradient can be computed efficiently using backpropagation.

Principles of Evasion Attacks

- The attack should work (evade the target model)
- It should be imperceptible.
 - This is very easy to quantify in images e.g., it still looks like a panda
 - It's common to use L-norms to quantify the amount of perturbation
 - This problem is *much* harder to quantify in software / malware / phishing
- It should be functional
 - If you are perturbing software, the resulting software should retain all the functionality of the original software
 - A picture should still look like the original picture
- Ideally the perturbations are *small* (to avoid detection).

Error Specific vs Generic

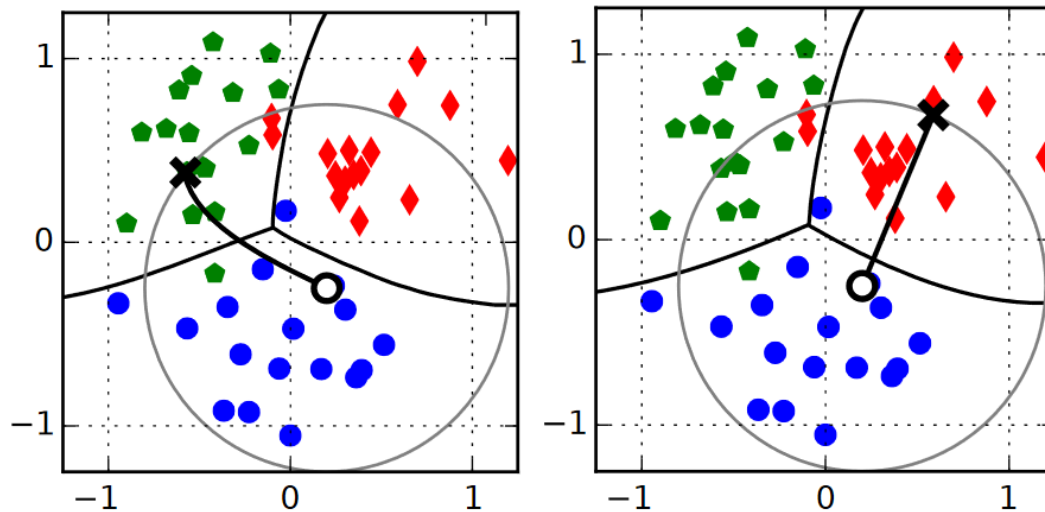


Figure 4: Examples of error-specific (*left*) and error-generic (*right*) evasion [6]. Decision boundaries among the three classes (blue, red and green points) are shown as black lines. In the error-specific case, the initial (blue) sample is shifted towards the green class (selected as target). In the error-generic case, instead, it is shifted towards the red class, as it is the closest class to the initial sample. The gray circle represents the feasible domain, given as an upper bound on the ℓ_2 distance between the initial and the manipulated attack sample.

Wild Patterns: Ten Years After the Rise of Adversarial Machine Learning

Battista Biggio^{a,b,*}, Fabio Roli^{a,b}

^aDepartment of Electrical and Electronic Engineering,
University of Cagliari,
Piazza d'Armi 09123 Cagliari, Italy

^bPluribus One, Cagliari, Italy

Wild Patterns: Ten Years After the Rise of Adversarial
Machine Learning
<https://arxiv.org/pdf/1712.03141.pdf>

Mimicry Attacks

- There was an earlier (parallel) idea in the security domain
- What if we made attacks that just looked more legitimate?
- Could they evade machine learning models?
- These attacks are known as mimicry attacks
- You basically update your input to be more like the target class (the benign class), while still retaining its maliciousness

Why is this even possible?

- It seems that if we update our inputs, to be like the outputs they should *turn* into the outputs!
- But instead we can give them little pushes and suddenly the model gets it wrong
- Ideas:
 - Our models are totally overfit
 - The decision boundaries are just not in a good place - they are too close to the input class, like a tight fence.
 - We are using features which don't work well (or something like that)

Conclusion

- General intuitions for how evasion attacks work
- Requirements of an evasion attack
 - imperceptible
 - functional
- Targeted vs untargeted attacks
- Mused about why it works

Place your
Webcam Video here
Size 100%



Threat Modeling Adversarial Evasion Attacks

Gilad Gressel
Amrita School of Engineering

Introduction

- Perfect Knowledge
- Limited Knowledge
- Zero Knowledge
- Kerckhoff's Principle

Place your
Webcam Video here
Size 100%

General Threat Models



Perfect
Knowledge

White Box

Limited
Knowledge

Grey Box

Zero
Knowledge

Black Box

Perfect Knowledge - White Box



- You know everything!
- Algorithm used to train defending model
 - hyper-parameters
 - model weights
- Training data
- Testing / Evaluation data
- Feature representation
- You just don't know the password to the users computer

Limited Knowledge – Grey Box



- You know ... something
- You don't know the feature representation, but you have access to some samples
- You know what kind of model is defending, but you don't the model weights (or hyper-parameters)
- You are able to collect some data that is probably very similar to the defending models data

Zero Knowledge - Black Box



- You don't know anything ... Except you do know
- What the model is defending or doing (malware detector, image classifier, etc.)
- You are able to query the model – imagine it is a cloud API, you can ask it questions and get back answers
- You *may* get model probabilities or hard labels
- You have to make your own assumptions about the feature representation of the model

Kerckhoff's Principle



- Kerckhoff's principle in cryptography states that all parts of the system should be public, with the exception of the private key
- This is opposite security through obscurity (hiding parts of your system)
- If we want machine learning models to defend well, they would withstand white box attacks



What attacks should be researched?

- Typically, researchers begin novel attacks with PK (white box) threat models and work *towards* a black box
- The intuition is that “white-box” attacks are easiest to do, since you know everything
- Black box attacks are more realistic though, since real defenses will not broadcast their secrets
- We are far from the ideal scenario!

Conclusion

- Different threat models
- Kerckchoff's principle in the adversarial setting



Place your
Webcam Video here
Size 100%



Zero Knowledge Black Box Evasion Attacks

Gilad Gressel
Amrita School of Engineering

Introduction

- Principle of transferability
- Surrogate Models
- Hard Labels vs Probabilities
- Assumptions

Place your
Webcam Video here
Size 100%

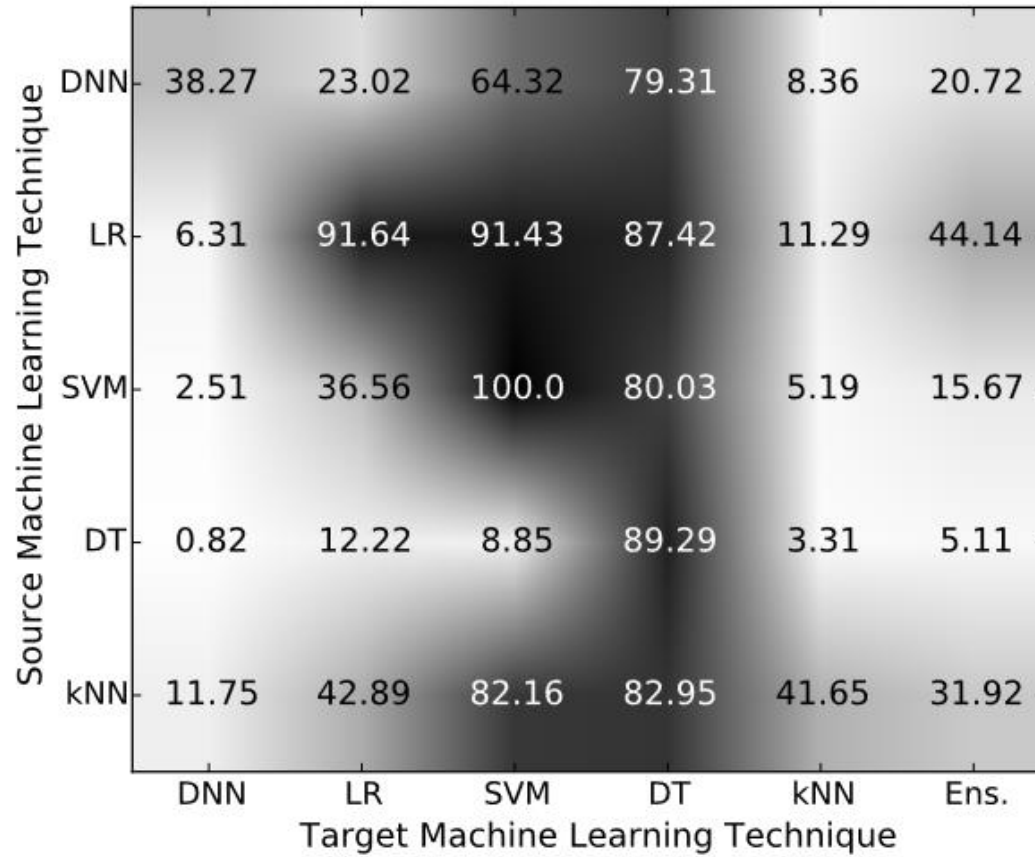


Figure 3: cross-technique Transferability matrix: cell (i, j) is the percentage of adversarial samples crafted to mislead a classifier learned using machine learning technique i that are misclassified by a classifier trained with technique j .

Principle of Transferability

Transferability in Machine Learning: from Phenomena to Black-Box Attacks using Adversarial Samples

Nicolas Papernot and Patrick McDaniel
The Pennsylvania State University
University Park, PA
{ngp5056,mcdaniel}@cse.psu.edu

Ian Goodfellow
OpenAI
San Francisco, CA
ian@openai.com

Surrogate Model



- Get a dataset on your own
- Query the target model you want to attack and get labels back
- Label your dataset
- Train a “surrogate” model locally with your newly annotated dataset
- Attack your local model with a white box attack
- “Transfer” the attack to the defending model

Hard Labels vs Probabilities

- Probabilities: ".85 likelihood it's a dog"
- Hard labels: "It's a dog"
- Probabilities make it *much* easier to calibrate your attack
- Hard labels make it harder
- Why then would a vendor give probabilities?
- Help users make decisions in uncertainty
- "We think this is suspicious" vs "you never found the email because we put it in the spambox"

Assumptions



- In all black box attacks, there is *some* assumption of the feature representation
- Most work has been done in vision – so we assume pixels are the features
- In security domains, this becomes *much* more complicated.

Conclusion

- Transferability
- Surrogate Models
- Hard labels vs Probabilities
- Feature Representation Assumptions

Place your
Webcam Video here
Size 100%



Heterogeneous vs Homogeneous Data

Gilad Gressel
Amrita School of Engineering

Introduction

- Homogeneous data
- Heterogenous (Tabular) data
- Feature Space vs Problem Space
- Examples

Place your
Webcam Video here
Size 100%

Homogeneous Datasets



- In a homogeneous dataset all the features are the same
- Example: a computer vision dataset
- A single sample is an image, each image has a single feature representation, the pixel
- A single image may have 28x28 pixels (784 total)
- The representation of each feature is identical – they are all continuous and bounded between 0-255
- While we have many features (columns), each one is semantically identical



Attacks on Homogeneous Datasets



- Evasion attacks are easy to perform against homogeneous datasets because all features can be manipulated in the same way
- We can use gradients naturally
- When we optimize an input image to evade a vision classifier we follow the gradient and update whichever pixels need to be updated.
- This works because pixels are continuous, and all equally allowed to be perturbed



Heterogeneous (Tabular) Datasets

- Heterogeneous (tabular) datasets have features which are *not* all the same:
 - Continuous - size of the house in square feet
 - Categorical - color of the house
 - Ordinal - # of bedrooms in the house
- Heterogeneous features may be immutable (the attacker cannot control it):
 - The age of a person
 - The domain of a website
- Features that require validity:
 - Someone's death date cannot be before their birthdate
 - Number of different API calls cannot exceed the server capacity

Challenges in Tabular Space



- You cannot drop-in a gradient based method for any non-continuous features
- You must obey individual feature constraints (validity and feasibility)
- How can we measure perceptibility in a tabular domain?
 - Perception in a picture is obvious to all humans (it's a dog)
 - Perception in a credit card fraud dataset? Only experts could tell you
- How do we guarantee functionality?

Mapping from Feature Space to Problem Space



- In a computer vision, the features *are* the problem space! A perturbation in pixels translations to a new picture
- The same is *not* true in tabular space
- A perturbed phishing site in feature space may not translate to a functional website
- This is known as the “inverse feature mapping problem”
 - It is easy to perturb samples in feature space
 - How do we go backwards into problem space?
 - How de we create an actual malware sample from the features?
 - There is *not* easy to do

Examples of Problem Space Attacks

- Phishing websites
- Malicious text
- Malicious PDF's
- Android malware
- Windows malware
- Malicious Javascript
- Network intrusion detection systems
- Industrial Control Systems
- Source code attribution

Conclusion

- Tabular vs Homogeneous datasets
- Challenges in Tabular datasets
- Inverse feature mapping problem

Place your
Webcam Video here
Size 100%